

Operating System

CS-2006

Lecture 4

Mahzaib Younas

Lecture Department of Computer Science
FAST NUCES CFD

Operating System Services

Outlines

- Operating System Services
- User and Operating System-Interface
- System Calls
- System Services
- Linkers and Loaders
- Why Applications are Operating System Specific
- Design and Implementation
- Operating System Structure
- Building and Booting an Operating System
- Operating System Debugging

Operating System Services

Operating systems provide an environment for execution of programs and services to programs and users.

Set of OS services provides functions that are helpful to the user

1. User interface
2. Program execution
3. I/O operations
4. File-system manipulation
5. Commination
6. Error Detection

set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing

1. Resource Allocation
2. Logging
3. Protection and security

User Services

User interface

- Almost all operating systems have a user interface (UI).
- Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **touch-screen**, **Batch**

Program execution

- The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)

Error Detection

- OS needs to be constantly aware of possible errors
- May occur in the CPU and memory hardware, in I/O devices, in user program
- For each type of error, OS should take the appropriate action to ensure correct and consistent computing
- Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

File-system manipulation

- The file system is of particular interest.
- Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

I/O operations

- A running program may require I/O, which may involve a file or an I/O device

Communication

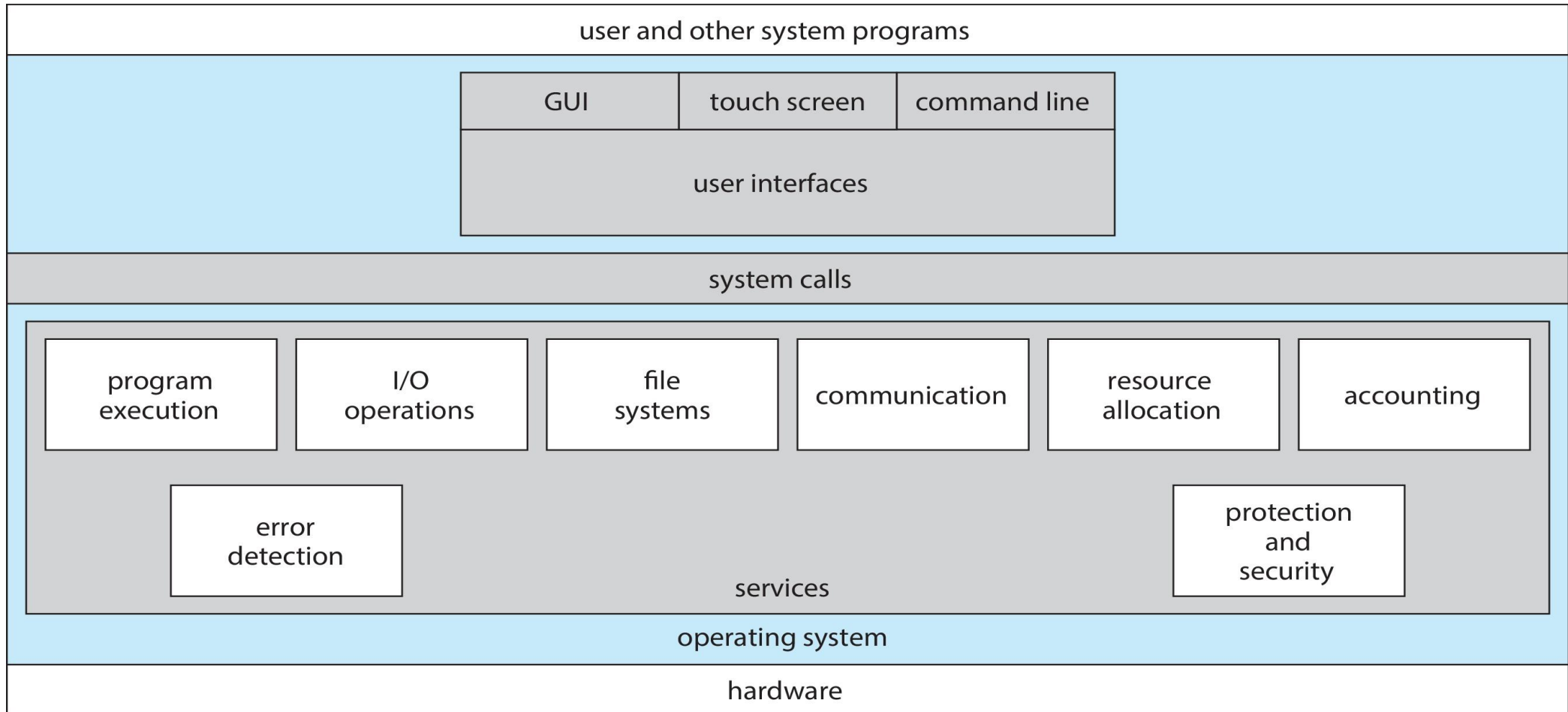
- Processes may exchange information, on the same computer or between computers over a network
- Communications may be via shared memory or through message passing (packets moved by the OS)

Resource Sharing

Resource allocation - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them

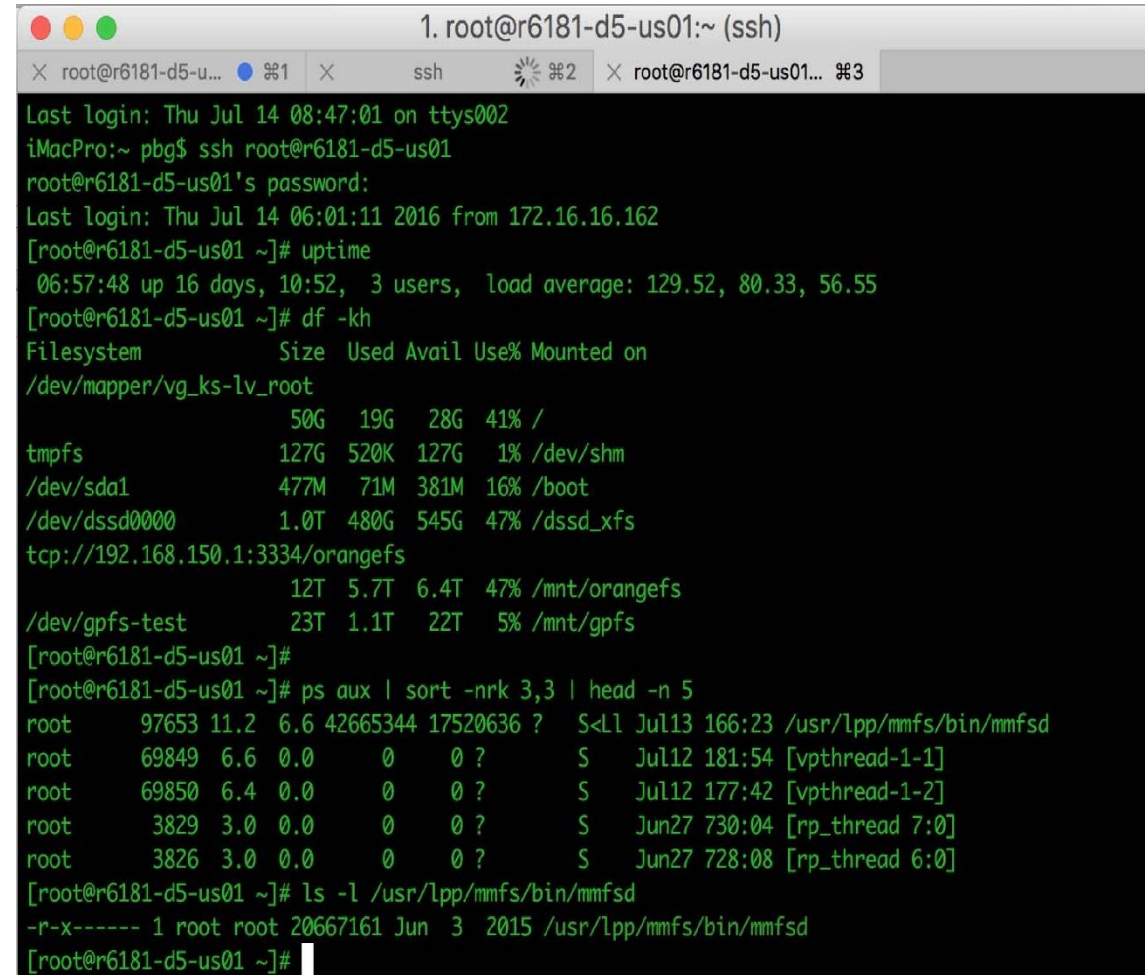
- Many types of resources - CPU cycles, main memory, file storage, I/O devices.
- **Logging** - To keep track of which users use how much and what kinds of computer resources
- **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - **Protection** involves ensuring that all access to system resources is controlled
 - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

A View of Operating System Services



Command Line Interpreter

- CLI allows direct command entry
 - Sometimes implemented in kernel, sometimes by systems program
 - Sometimes multiple flavors implemented – shells
 - Primarily fetches a command from user and executes it
 - Sometimes commands built-in, sometimes just names of programs
 - If the latter, adding new features doesn't require shell modification



```
1. root@r6181-d5-us01:~ (ssh)
X root@r6181-d5-u... 1 X ssh 2 X root@r6181-d5-us01... 3
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pb$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root
                  50G   19G   28G  41% /
tmpfs            127G  520K  127G   1% /dev/shm
/dev/sda1        477M   71M  381M  16% /boot
/dev/dssd0000    1.0T  480G  545G  47% /dssd_xfs
tcp://192.168.150.1:3334/orangefs
                  12T   5.7T   6.4T  47% /mnt/orangefs
/dev/gpfs-test   23T   1.1T   22T   5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root    97653 11.2  6.6 42665344 17520636 ?    S<Ll  Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root    69849  6.6   0.0      0      0 ?        S    Jul12 181:54 [vpthread-1-1]
root    69850  6.4   0.0      0      0 ?        S    Jul12 177:42 [vpthread-1-2]
root     3829  3.0   0.0      0      0 ?        S    Jun27 730:04 [rp_thread 7:0]
root     3826  3.0   0.0      0      0 ?        S    Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x----- 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#
```


Graphical User Interface

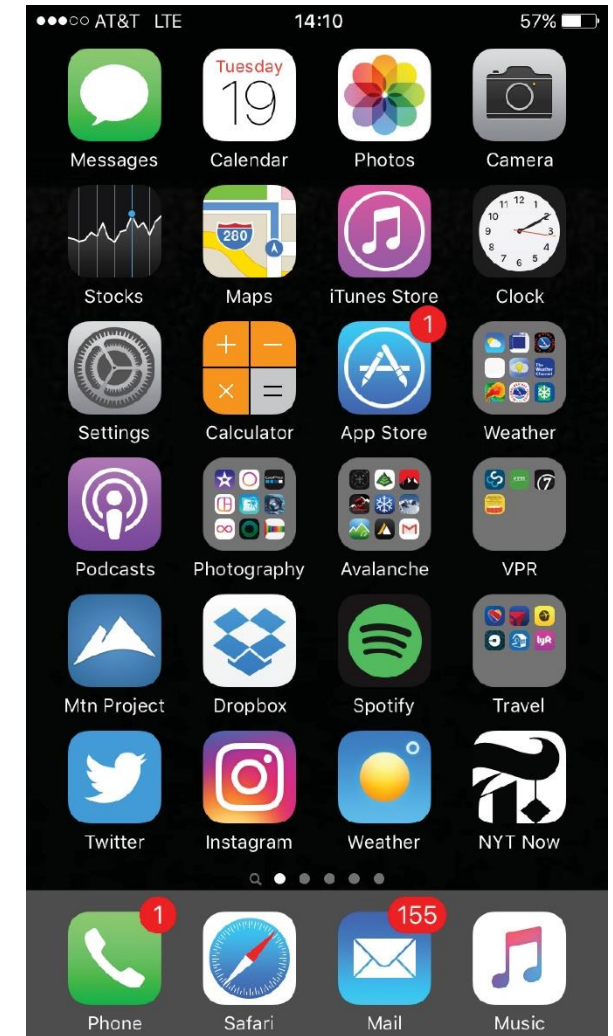
- User-friendly desktop metaphor interface
 - Usually mouse, keyboard, and monitor
 - Icons represent files, programs, actions, etc
 - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a folder))
- Invented at Xerox PARC

Both GUI and CLI

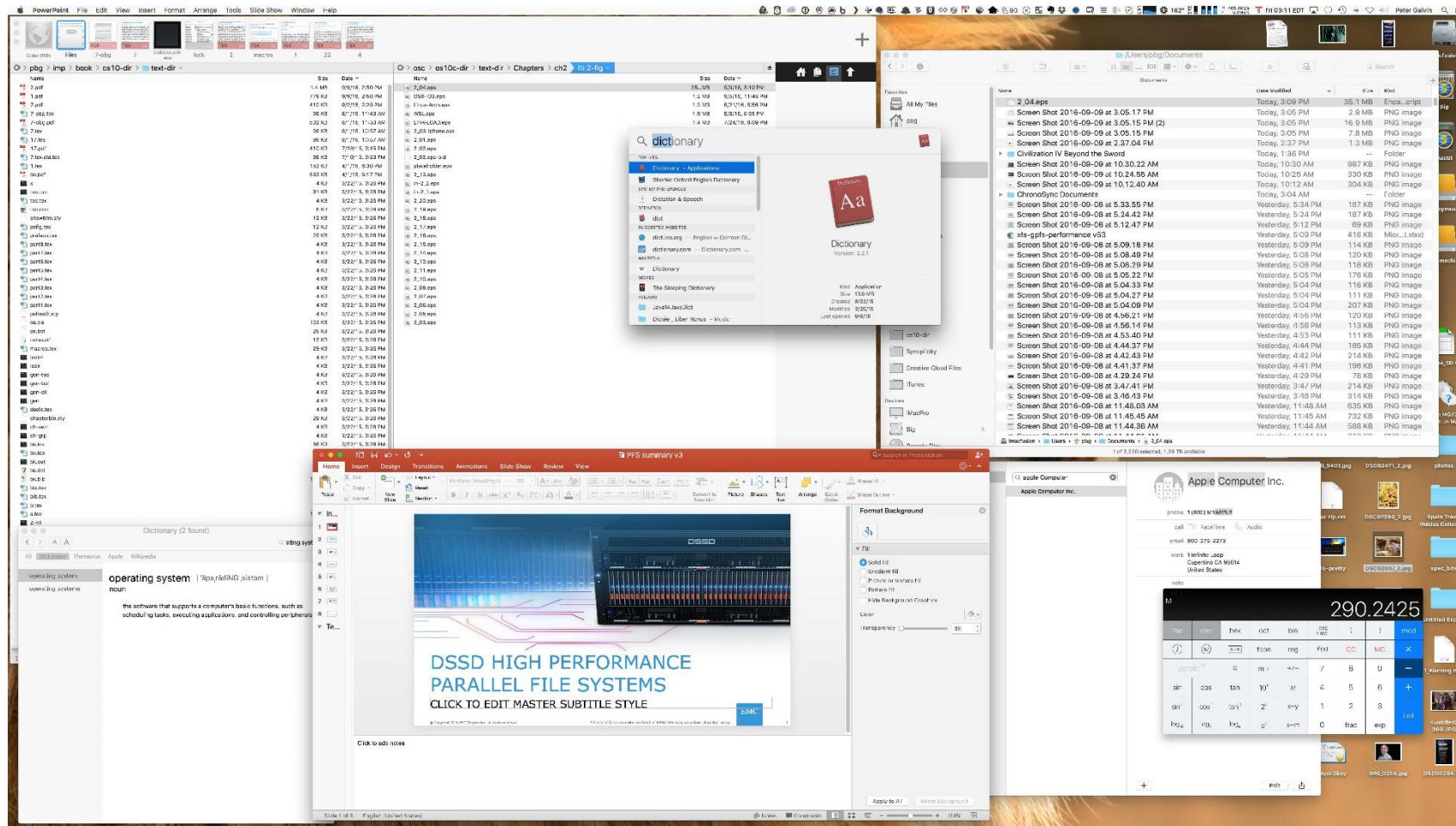
- Many systems now include both CLI and GUI interfaces
 - Microsoft Windows is GUI with CLI “command” shell
 - Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
 - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

Touch Screen Interface

- Touchscreen devices require new interfaces
 - Mouse not possible or not desired
 - Actions and selection based on gestures
- Virtual keyboard for text entry
- Voice commands



The MAC OS X GUI

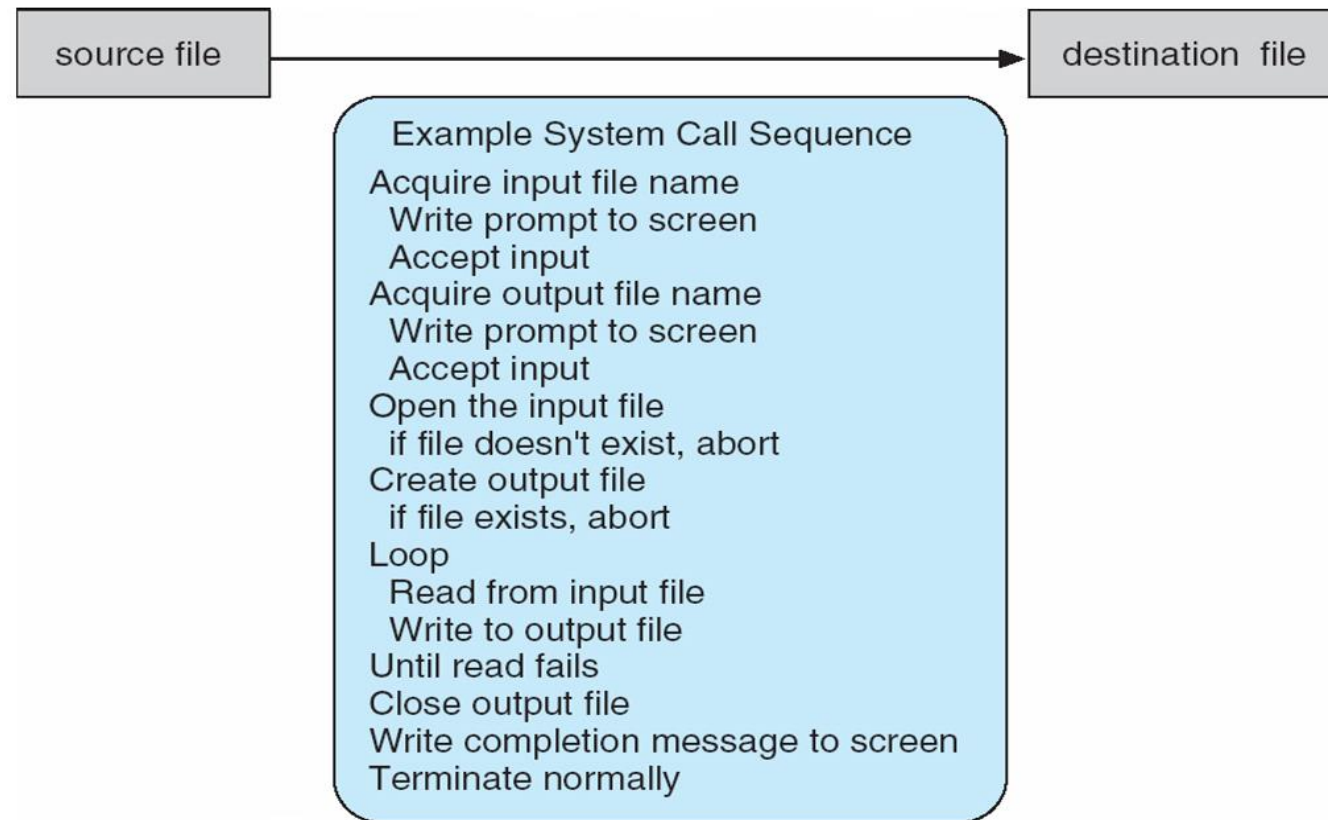


System Calls

- Programming interface to the services provided by the OS
 - Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level Application Programming Interface (API) rather than direct system call use
- Three most common APIs are
 1. Win32 API for Windows
 2. POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X)
 3. Java API for the Java virtual machine (JVM)

Example of system calls

- System call sequence to copy the content of one file to other



Example of standard APIs

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

<pre>#include <unistd.h></pre>		
<pre>ssize_t</pre>	<pre>read(int fd, void *buf, size_t count)</pre>	
return value	function name	parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

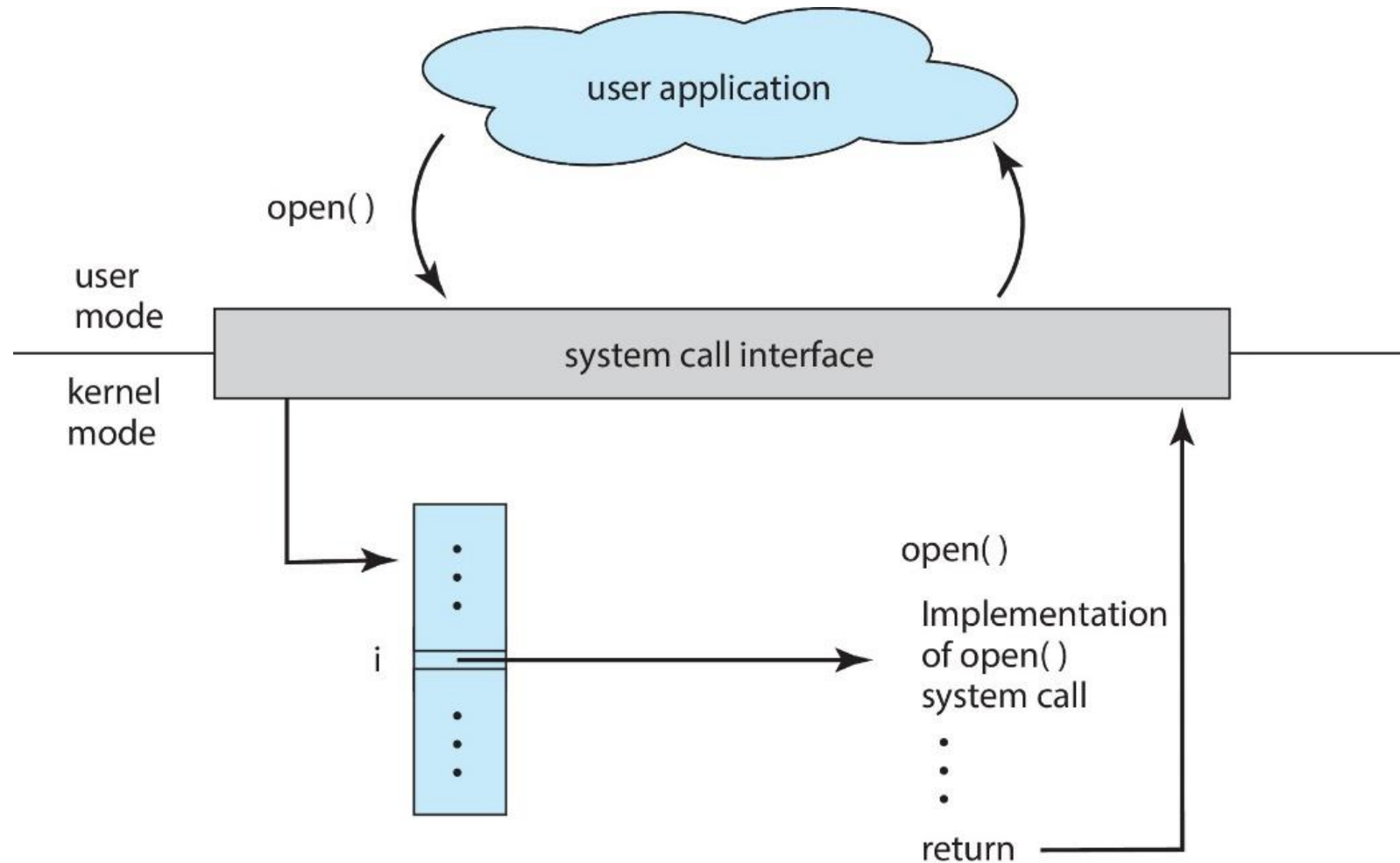
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

Systems Call Implementation

- Typically, a number is associated with each system call
 - **System-call interface maintains a table indexed according to these numbers**
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result call
 - Most details of OS interface hidden from programmer by API
 - Managed by run-time support library (set of functions built into libraries included with compiler)

API – system call OS relation



System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
 - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
 - Simplest: pass the parameters in registers
 - In some cases, may be more parameters than registers
 - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
 - This approach taken by Linux and Solaris
 - Parameters placed, or pushed, onto the stack by the program and popped off the stack by the operating system
 - Block and stack methods do not limit the number or length of parameters being passed